

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	S. Jaswanth	Reg. No.:	192525214
Section / Batch:	AIML	Date:	20/07/2026

Code of Conduct

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

Q1. Insert the following elements 150, 200, 250, 400, 550 into the Doubly linked and delete the elements 150 and 250. Show the linked structure before and after delete operations.

Step 1:- Insert Elements

After inserting all elements:

Step 2: Delete 150 (Head Node)

Update:

- Head = 250
- 250, prev = NULL

Step 3: Delete 250 (new Head)

Update:

- Head = 350
- 350, prev = NULL

Final Doubly linked list

Insertion

1. Create a New node.
2. If list is empty, make it the head.
3. Otherwise, traverse to the last ~~head~~ node.
4. Set last $\rightarrow$ next = newNode and newNode $\rightarrow$ prev = last.

Deletion

1. Search for the node
2. If it's the head, move head to the next node.
3. Update previous and next links.
4. Free the deleted node.

Time Complexity

- Insertion at end : $O(n)$
- Deletion by Value : $O(n)$

Q2. Circular Queue (Size = 6)

Operations:

Assume array index = 0 to 5.

Operation	Queue	front	Rear
initial	Empty	-1	-1
Enqueue(10)	10	0	0

Enqueue (20)	10, 20	0	1
Enqueue (30)	10, 20, 30	0	2
Dequeue (1)	20, 30	1	2
Enqueue (40)	20, 30, 40	1	3
Enqueue (50)	20, 30, 40, 50	1	4
Enqueue (60)	20, 30, 40, 50, 60	1	5
Enqueue (70)	20, 30, 40, 50, 60, 70	1	0

Final Circular Queue

Index	0	1	2	3	4	5
Value	70	20	30	40	50	60

Front = 1 (Element = 20)

Rear = 0 (Element = 70)

Explanation:

- Enqueue: Inserts an element at the rear.
If the rear reaches the last index, it wraps to index 0 using $(\text{rear} + 1) \% \text{size}$.

- Dequeue: Remove the front element and advance the front pointer using $(\text{front} + 1) \% \text{size}$.

- Advantage of Circular Queue:

It efficiently reuses empty spaces created by deletions, avoiding wasted memory and the false overflow seen in a linear queue.

WPS